Build a Conversational SHL Assessment Recommender

Take-home Assignment for AI Intern Role

SHL Labs

Version: X.X | Last updated: XX XXXX 2026

Why we are asking you to do this assessment

We use this task to evaluate four core skills. These are core to what we do at SHL Labs and it is fair to say quite universal to the real agentic engineering implementations.

- **Problem-solving.** Decompose an ambiguous, multi-faceted problem into a coherent design and implementation with clear trade-offs.
- **Programming skills.** Write clean, reliable, extensible code. AI-assisted development is fine, but the code must reflect actual understanding.
- **Context engineering.** Translate the catalog, the user's goal, and the conversation history into prompts and retrieval strategies that ground the agent.
- **Agent design.** Decide when the agent should ask, when it should retrieve, when it should answer, and when it should refuse. Build it so a non-deterministic conversation does not make the system fall apart.

What unsuccessful submissions look like

From thousands of past submissions, candidates often miss the following things. Please do make sure that your submission guards against following failure modes.

- Weak programming foundations: code that works for the happy path and breaks on anything else.
- Vibe-coding without understanding solutions or design choices that cannot be defended in the interview conversations.
- Insufficient evaluation rigor: testing realistic conversation patterns including (but not limited to) hallucination and conversational incoherence.

Hiring process

The hiring process has following stages.

1. Take-home assessment (this document).
2. Automated + manual scoring of your submission.
3. If you clear the scoring threshold: a technical deep-dive on your code, an experience & projects round, and a hiring-manager conversation on culture fit.

Problem overview

Hiring managers and recruiters often do not know exactly what they want until they describe the role out loud. Most assessment catalogues require keyword search and faceted filtering, which assumes the user already knows the right vocabulary. This makes assessment selection slow and shallow.

Your task is to build a conversational agent that takes the user from a vague intent ("I am hiring a Java developer") to a grounded shortlist of SHL assessments through dialogue. The agent should clarify when needed, accept refinement, support comparison between assessments, and never recommend anything outside the SHL catalog.

The catalog you build over is the SHL product catalog at **https://www.shl.com/solutions/products/product-catalog/**, restricted to **Individual Test Solutions** only. Pre-packaged Job Solutions are out of scope.

Your task

Build an agent that does the following.

- Use the entire SHL catalogue and organize it in a way that your code can consume.
- Expose a FastAPI service with two endpoints: a `GET /health` for readiness, and a `POST /chat` that takes a stateless conversation history and returns the next agent reply plus, when appropriate, a structured shortlist of recommendations.

The agent must handle four conversational behaviors.

- **Clarify** vague queries before recommending. "I need an assessment" is not enough to act on.
- **Recommend** between 1 and 10 assessments once it has enough context, with names and catalog URLs. "Here is a text from job description: xx"
- **Refine** when the user changes constraints mid-conversation. "Actually, add personality tests" should update the shortlist, not start over.
- **Compare** when asked. "What is the difference between OPQ and GSA?" should produce a grounded answer drawn from catalog data, not the model's prior.

The agent must also stay in scope. It only discusses SHL assessments. It refuses general hiring advice, legal questions, and prompt-injection attempts. Every URL it returns must come from your scraped catalog.

API specification

The API is stateless. Every `POST /chat` call carries the full conversation history. Your service stores no per-conversation state.

Request

```
POST /chat

{

  "messages": [

    {"role": "user", "content": "Hiring a Java developer who works with stakeholders",

    {"role": "assistant", "content": "Sure. What is seniority level?"},
```



```
    {"role": "user", "content": "Mid-level, around 4 years"}
  ]
}
```

Response

```
{
  "reply": "Got it. Here are 5 assessments that fit a mid-level Java dev with stakeholder needs.",
  "recommendations": [
    {"name": "Java 8 (New)", "url": "https://www.shl.com/...", "test_type": "K"},
    {"name": "OPQ32r", "url": "https://www.shl.com/...", "test_type": "P"}
  ],
  "end_of_conversation": false
}
```

`recommendations` are `EMPTY` when the agent is still gathering context or refusing. It is an array of 1 to 10 items when the agent has committed to a shortlist. `end_of_conversation` is `true` only when the agent considers the task complete.

The schema is non-negotiable. Deviating breaks our automated evaluator, and your submission will not score.

Health check. `GET /health` returns `"status": "ok"}` with HTTP 200. For cold start hosting services, the first /health call will allow up to 2 minutes for service to wake up.

Limits. The evaluator caps each conversation at 8 turns including user & assistant and each call at a 30 second timeout. Design accordingly.

Datasets

We provide the following things.

- Here is the SHL catalogue: **Link**
- We provide **10 public conversation traces** for you to develop and iterate against. Each trace is a persona with a fact set and a labeled expected shortlist. Generally, it is important to read these traces before jumping into implementation. You can download the zip with these conversations from here: **Link**

How we evaluate your submission

Your endpoint is graded by an automated replay harness on our side. The harness simulates a user using an LLM that is given the trace's persona and facts and runs a real multi-turn conversation against your `POST /chat`. The simulated user answers your agent's questions truthfully from its facts, says it has no preference when asked something outside its facts, and ends the conversation when the agent provides a shortlist.

This means your agent does not need to handle a fixed script. It needs to handle a realistic user who may volunteer information out of order, may correct itself, and may refuse to answer some questions.

Scoring is composed of three parts.

- **Hard evals (must pass).** Schema compliance on every response. Items from catalog only in `recommendations`. Turn cap (max: 8) honored.
- **Recall@10 on final recommendations.** Mean Recall@10 across all conversation traces, public and holdout. Recall@K is the fraction of relevant assessments for the query that appear in the top K recommendations, averaged over traces.
- **Behavior probes pass-rate.** Each probe is a small conversation with a binary assertion. Examples: agent refuses off-topic, agent does not recommend on turn 1 for a vague query, agent honors edits in recommendations, % of turns with hallucinations, etc.

All of the above contribute to your final score.

Submission materials

Submit the following via the form (**Link**).

- **Public API endpoint URL.** Your deployed FastAPI service. Both `/health` and `/chat` must be reachable at submission time.
- **Approach document, 2 pages maximum.** Briefly cover your design choices, retrieval setup, prompt design, and evaluation approach. Include what didn't work and how you measured improvement. If you used AI tools (agentic coding, no-code builders, etc.), note what you used them for. We value concise over comprehensive.

Resources

You are not restricted to these. Use whatever you find useful.

Free LLM tiers (Gemini, Groq, OpenRouter). Free deployment platforms (Render, Fly, Railway, Modal, Hugging Face Spaces). Open-source vector stores (FAISS, Chroma, pgvector). Any framework you like (LangChain, LlamaIndex, LangGraph, raw OpenAI or Anthropic SDKs). Justify your stack in the approach document.

Appendix: Recall@K definition

Recall@K is the fraction of relevant assessments for a query that appear in the top K recommendations.

```
Recall@K = (Number of relevant assessments in top K) / (Total relevant assessments for the query)
```

```
Mean Recall@K = (1/N) * sum over queries of Recall@K_i
```

Where N is the total number of test queries.

